📺 OLED Display

Hardware Overview

📺 What is it?

The **OLED** (Organic Light Emitting Diode) display is the primary visual output. Unlike traditional LCD screens that require a bulky backlight, every single pixel on an OLED screen is an independent microscopic LED that produces its own light

⚡ Electrical Explanation

The screen is controlled via a communication protocol called **I²C** (Inter-Integrated Circuit). Instead of needing `8,192` individual wires to control every pixel, I²C allows the microcontroller to send thousands of commands per second over just two wires:

1. **`SDA` (Data):** Transmits the actual `1`s and `0`s of the picture data.
2. **`SCL` (Clock):** A metronome that keeps the sender and receiver perfectly synchronized.

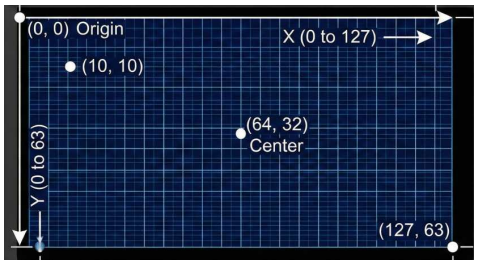
Key Hardware Facts:

- **Resolution: `128` pixels wide** by **`64` pixels high**.
- **Colors:** 1-Bit Monochrome. A pixel can only exist in two states: ON (White) or OFF (Black).

🧩 Coordinate System

The screen works like a grid.

- The Origin **`(0, 0)`** is the **Top-Left** corner.
- **`X`** increases as you move to the **Right** (`0` to **`127`**).
- **`Y`** increases as you move **Down** (`0` to **`63`**).



Software & Programming

📚 `kcomp_oled.h`

Instead of the heavy libraries, we use a custom library designed specifically for the ATmega328P.
Usage: `#include "kcomp_oled.h"`

🧠 The Framebuffer (The "Shadow Screen")

Writing to the screen takes time. If we updated the screen every time we drew something, it flickers.
The Solution: K-Comp creates a "virtual screen" in its memory called a **Buffer**. How it works:

1. You draw shapes and text to the **buffer**.
2. The real screen remains blank or shows the old image.
3. You call `updateDisplay()` to **push** the buffer to the real screen.

Use `OLED_COLOR_WHITE`, `OLED_COLOR_BLACK`, or `OLED_COLOR_INVERT` for `color` parameters.

⚙️ Function Reference - OLED Control Functions

Function Name	Description & Parameters
`bool initDisplay()`	**Setup:** Wakes up the screen, configures the charge pump, and clears memory. ▪ **Returns:** `true` if successful, `false` if missing.
`void clearDisplay()`	**Erase:** Wipes the memory buffer clean (sets all pixels to black). **Always call this when drawing something new!**
`void updateDisplay()`	**Show:** Pushes the drawing from memory to the actual screen. **Nothing appears until you call this.**
`void invertDisplay(bool invert)`	**Silicon Invert:** Hardware command to instantly swap Black and White. ▪ `invert`: `true` to invert colors, `false` for normal display.
`void dimDisplay(bool dim)`	**Hardware Dimming:** Drops physical voltage to dim the screen and save power. ▪ `dim`: `true` to dim, `false` for full brightness.

🛠️ Function Reference - Geometry Engine (Shapes)

Function Name	Description & Parameters
`void drawPixel(int16_t x, int16_t y, uint8_t color)`	**Dot:** Turns on a single pixel. ▪ `x`: Horizontal coordinate (`0` to `127`). ▪ `y`: Vertical coordinate (`0` to `63`). ▪ `color`: Pixel color state.
`void drawLine(int16_t x0, int16_t y0, int16_t x1, int16_t y1, uint8_t color)`	**Line:** Draws a diagonal line. ▪ `x0`, `y0`: Starting coordinates. ▪ `x1`, `y1`: Ending coordinates. ▪ `color`: Pixel color state.
`void drawFastVLine(int16_t x, int16_t y, int16_t h, uint8_t color)`	**Vertical Line:** High-speed line. ▪ `x`, `y`: Starting top coordinate. ▪ `h`: Total height in pixels. ▪ `color`: Pixel color state.
`void drawFastHLine(int16_t x, int16_t y, int16_t w, uint8_t color)`	**Horizontal Line:** High-speed line. ▪ `x`, `y`: Starting left coordinate. ▪ `w`: Total width in pixels. ▪ `color`: Pixel color state.
`void drawRect(int16_t x, int16_t y, int16_t w, int16_t h, uint8_t color)`	**Box (Outline):** Draws an empty rectangle. ▪ `x`, `y`: Top-left corner coordinate. ▪ `w`: Width of the rectangle. ▪ `h`: Height of the rectangle. ▪ `color`: Line color state.
`void fillRect(int16_t x, int16_t y, int16_t w, int16_t h, uint8_t color)`	**Box (Solid):** Draws a filled rectangle. ▪ `x`, `y`: Top-left corner coordinate. ▪ `w`: Width of the rectangle. ▪ `h`: Height of the rectangle. ▪ `color`: Fill color state.

	Circle (Outline): Draws a hollow circle. ▪ `x0`, `y0`: Center coordinate of the circle. ▪ r: Radius in pixels. ▪ color: Line color state.
`void drawCircle(int16_t x0, int16_t y0, int16_t r, uint8_t color)`	**Circle (Solid):** Draws a filled circle. ▪ `x0`, `y0`: Center coordinate of the circle. ▪ `r`: Radius in pixels. ▪ color: Fill color state.
`void fillCircle(int16_t x0, int16_t y0, int16_t r, uint8_t color)`	**Circle (Solid):** Draws a filled circle. ▪ `x0`, `y0`: Center coordinate of the circle. ▪ `r`: Radius in pixels. ▪ `color`: Fill color state.
`void drawTriangle(int16_t x0, int16_t y0, int16_t x1, int16_t y1, int16_t x2, int16_t y2, uint8_t color)`	**Triangle (Outline):** Draws a hollow triangle. ▪ `x0`, `y0`: First point. ▪ `x1`, `y1`: Second point. ▪ `x2`, `y2`: Third point. ▪ `color`: Line color state.
`void fillTriangle(int16_t x0, int16_t y0, int16_t x1, int16_t y1, int16_t x2, int16_t y2, uint8_t color)`	**Triangle (Solid):** Draws a filled triangle. ▪ `x0`, `y0`: First point. ▪ `x1`, `y1`: Second point. ▪ `x2`, `y2`: Third point. ▪ `color`: Fill color state.

Function Reference - Text & Bitmap Engine

Function Name	Description & Parameters
`void setCursor(int16_t x, int16_t y)`	**Position:** Sets the starting coordinate. ▪ `x`: Horizontal starting point (left edge of text). ▪ `y`: Vertical starting point (top edge of text).
`void printChar(char c, uint8_t color, uint8_t size)`	**Write Character:** Renders a single character. ▪ `c`: The character to print (e.g., `'A'`). ▪ `color`: Character color state. ▪ `size`: Text scaling multiplier. (`OLED_TEXT_NORMAL`, `OLED_TEXT_LARGE`, `OLED_TEXT_HUGE`).
`void printText(const char* text, uint8_t color, uint8_t size)`	**Write String:** Renders text. ▪ `text`: The string to print (e.g., `"HELLO"`). ▪ `color`: Text color state. ▪ `size`: Text scaling multiplier (`OLED_TEXT_NORMAL`, `OLED_TEXT_LARGE`, `OLED_TEXT_HUGE`).
`void printInt(long value, uint8_t color, uint8_t size)`	**Write Integer:** Renders a whole number. ▪ `value`: The number to print. ▪ `color`: Character color state.

	▪ `size`: Text scaling multiplier. (`OLED_TEXT_NORMAL`, `OLED_TEXT_LARGE`, `OLED_TEXT_HUGE`).
`void printFloat(float value, uint8_t decimalPlaces, uint8_t color, uint8_t size)`	**Write Decimal:** Renders a floating-point number. ▪ `value`: The decimal number to print. ▪ `decimalPlaces`: Number of decimal places. ▪ `color`: Character color state. ▪ `size`: Text scaling multiplier. (`OLED_TEXT_NORMAL`, `OLED_TEXT_LARGE`, `OLED_TEXT_HUGE`).
`void drawBitmap(int16_t x, int16_t y, const uint8_t *bitmap, int16_t w, int16_t h, uint8_t color)`	**Image:** Renders a `1`-bit array stored in `PROGMEM`. ▪ `x`, `y`: Top-left corner coordinate. ▪ `bitmap`: The name of the byte array. ▪ `w`: Width of the graphic in pixels. ▪ `h`: Height of the graphic in pixels. ▪ `color`: Color to draw the `ON` pixels.

✨ Zero-Cost Layout Macros

These replace manual math. The compiler evaluates them instantly before the program runs.

Macro Signature	Description & Parameters
`CURRENT_WIDTH`	`128` or `64` depending on the current screen orientation.
`CURRENT_HEIGHT`	`128` or `64` depending on the current screen orientation.
`OLED_PCT_X(percent)`	**Calculate:** Returns the exact X pixel coordinate at a given percentage of screen width. ▪ `percent`: Integer from `0` to `100`.
`OLED_PCT_Y(percent)`	**Calculate:** Returns the exact Y pixel coordinate at a given percentage of screen height. ▪ `percent`: Integer from `0` to `100`.
`OLED_TEXT_WIDTH(charCount, size)`	**Calculate:** Returns the total horizontal pixel width of a text string based on the `6`-pixel character footprint. ▪ `charCount`: Total number of characters in your string. ▪ `size`: The text size multiplier used.
`OLED_TEXT_HEIGHT(size)`	**Calculate:** Returns the total vertical pixel height of a text string based on the 8-pixel character footprint. ▪ `size`: The text size multiplier used.
`OLED_ALIGN_CENTER_X(charCount, size)`	**Position:** Calculates the X-coordinate to center text horizontally. ▪ `charCount`: Total number of characters in your string. ▪ `size`: The text size multiplier used.
`OLED_ALIGN_RIGHT_X(charCount, size)`	**Position:** Calculates the X-coordinate needed to flush-right align text against pixel `127`. ▪ `charCount`: Total number of characters in your string. ▪ `size`: The text size multiplier used.
`OLED_ALIGN_CENTER_Y(size)`	**Position:** Calculates the exact Y-coordinate to center text vertically on the screen.

	▪ `size`: The text size multiplier used.
`OLED_ALIGN_BOTTOM_Y(size)`	**Position:** Calculates the Y-coordinate to flush-bottom align text so the letters rest on the bottom edge (pixel `63`). ▪ `size`: The text size multiplier used.

⚠️ Pitfalls & Troubleshooting

- **The Invisible Drawing** If your screen is completely black, check if you called `updateDisplay()` at the end of your loop. Drawing to the buffer does nothing to the physical screen until it is pushed!
  The Fix: Call `updateDisplay()` when you want something to be drawn on the screen.
- **The Memory Crash** Because the standard library requires a `1024`-byte buffer, it consumes half of the ATmega328P's available RAM before you even write a single line of logic. If your program behaves weirdly, resets randomly, or crashes when adding too many variables, you have likely hit the hardware memory ceiling.
  The Fix: You must switch your architecture to the **Paged Library** by using `#include <kcomp_oled_paged.h>`. The paged library slices the buffer into `8` horizontal bands, shrinking the memory footprint from `1024` bytes down to a `128` bytes. This frees up `896` bytes of RAM, giving your sensor logic and variables room to breathe.

📝 Sample Program

This program draws a centered title and an expanding circle.

```
#include <kcomp_oled.h>

void setup() {
  Serial.begin(115200);
  if (!initDisplay()) {
    Serial.println("OLED not found!");
    while(1);
  }
}

void loop() {
  int radius = 2;

  clearDisplay(); // 1. Erase the virtual buffer

  // 2. Draw text centered at the top
  // 11 characters
  setCursor(OLED_ALIGN_CENTER_X(11, 1), 5);
  printText("HELLO WORLD", OLED_COLOR_WHITE, 1);

  // 3. Draw a growing circle in the center of the screen
  drawCircle(64, 32, radius, OLED_COLOR_WHITE);

  radius++;
  if (radius > 20) radius = 2;

  updateDisplay(); // 4. Push everything to the physical screen!

  delay(50);
}
```

💡 **Try it out:** You can find a full sample program **`SimpleOLED`** in the K-Comp examples folder!

🗂️ The Paged Architecture (`kcomp_oled_paged.h`) only of relevance if you run out of RAM!

As your programs grow, `kcomp_oled.h` may trigger a memory crash because its `1024`-byte buffer consumes half of the ATmega328P's available RAM.

`kcomp_oled_paged.h` is a paged version of the `kcomp_oled.h` . Instead of keeping all `8192` pixels in memory at the same time, it slices the `128x64` screen into `8` horizontal bands (pages). Each page is `128` pixels wide and `8` pixels high. This shrinks the memory footprint down to `128` bytes.

Usage: `#include "kcomp_oled.h"`

🪢 Paging Concept Illustration

The diagram below shows how the `128x64` coordinate system is segmented into `8` independent, stacked memory bands:

```
                    X-Coordinate  (0 to 127) ➔
          ┌──────────────────────────────────────────────────────────┐
Page 0    │ Y: 0 to 7   [ Holds exactly 128 x 8 bits = 128 Bytes ]   │
          ├──────────────────────────────────────────────────────────┤
Page 1    │ Y: 8 to 15  [ Holds exactly 128 x 8 bits = 128 Bytes ]   │
          ├──────────────────────────────────────────────────────────┤
Page 2    │ Y: 16 to 23 [ Holds exactly 128 x 8 bits = 128 Bytes ]   │
          ├──────────────────────────────────────────────────────────┤
Page 3    │ Y: 24 to 31 [ Holds exactly 128 x 8 bits = 128 Bytes ]   │
          ├──────────────────────────────────────────────────────────┤
Page 4    │ Y: 32 to 39 [ Holds exactly 128 x 8 bits = 128 Bytes ]   │
          ├──────────────────────────────────────────────────────────┤
Page 5    │ Y: 40 to 47 [ Holds exactly 128 x 8 bits = 128 Bytes ]   │
          ├──────────────────────────────────────────────────────────┤
Page 6    │ Y: 44 to 55 [ Holds exactly 128 x 8 bits = 128 Bytes ]   │
          ├──────────────────────────────────────────────────────────┤
Page 7    │ Y: 56 to 63 [ Holds exactly 128 x 8 bits = 128 Bytes ]   │
          └──────────────────────────────────────────────────────────┘
```

⚠️ The Rules of the Page Loop

Because only one `128`-byte slice exists in RAM at a time, the microcontroller must redraw your entire screen **8** times per frame (once for each page). This introduces two critical rules:

- **The Frame Loop:** All of your geometry drawing and text printing functions *must* live inside the `firstPage()` / `nextPage()` loop.
- **Pure Determinism:** You must never change variables or read sensors inside the page loop. Because the loop repeats `8` times per frame, a command like `score++` inside the loop would erroneously add `8` points every single frame! Always handle your logic completely outside the display loop.
- **`clearDisplay` and `updateDisplay` do not exist anymore!**

⚙️ Function Reference - Paged Loop Control Functions

Function Name	Description & Parameters

`void firstPage()`	**Loop Anchor:** Resets the internal page pointer to page `0`. Always call this right before entering your render block.
`bool nextPage()`	**Frame Advance:** Pushes the current memory page to the physical screen and moves focus to the next band. ▪ **Returns:** `true` if more pages need rendering, `false` once all `8` pages are drawn.

Note: All standard shapes (`drawLine`, `drawCircle`) and text engine functions (`setCursor`, `printText`) operate exactly the same way inside the paged loop.

📝 Sample Program (Paged Architecture)

This program draws a centered title and an expanding circle using the memory-saving paged loop architecture.

```
#include <kcomp_oled_paged.h>  // Use the paged library header

int radius = 2;  // Global state variables stay safely outside the loops

void setup() {
  Serial.begin(115200);
  if (!initDisplay()) {  // Setup: Initializes the screen
    Serial.println("OLED not found!");
    while(1);
  }
}

void loop() {
  // --- RENDERING LAYER ---
  // The controller runs this block 8 times to construct the full frame
  firstPage();
  do {
    // Draw text centered horizontally at the top
    setCursor(OLED_ALIGN_CENTER_X(11, 1), 5);
    printText("HELLO WORLD", OLED_COLOR_WHITE, 1);

    // Draw the growing circle in the center
    drawCircle(64, 32, radius, OLED_COLOR_WHITE);

    // CRITICAL: Do NOT modify variables or counters here!

  } while (nextPage());  // Pushes the slice data to the physical screen

  // --- LOGIC LAYER (SAFE ZONE) ---
  // Modify your variables exactly ONCE per complete frame update
  radius++;
  if (radius > 20)
    radius = 2;
  }

  delay(50);
}
```

💡 **Try it out:** You can find a full sample program **`SimplePagedOLED`** in the K-Comp examples folder!